SE 216 – SOFTWARE PROJECT MANAGEMENT
PROJECT RISKS DOCUMENT

PROJECT NAME: Vaccine Integrity Monitoring System (VIMS)

GROUP MEMBERS: Muhammet Ali ÖKAN / Ceren ACAY / Azra AKGÜN / Emir Taha GÖKÇE / Ömer Talha ÇEKMEZ / Kayra Alp UŞAKLI / Hakkı Can DURAN / Ege ENGİN

LIKELIHOOD RANK	RISK DESCRIPTION
1	**Dependency Conflict & Version Incompatibility** - Conflicts between external libraries (e.g., IoT messaging clients and the core backend framework) during the build process could cause compilation errors, requiring significant time to resolve dependencies before feature development can continue.
2	**AI Training Data Scarcity -** Finding an accurate historical dataset of temperature fluctuations to train the AI predictive model might be difficult.
3	**Inadequate Testing Environment -** It will be difficult to test the system realistically without access to an actual logistics network or constant physical network drops.
4	**Asynchronous Data Handling & Queue Processing Errors -** Developing the backend logic to process high-frequency, continuous data streams asynchronously might lead to out-of-order packet processing, memory leaks, or data loss within the software's message broker/queue architecture.
5	**Key Personnel Turnover -** A critical team member drops out or becomes unavailable, leaving the rest of the team struggling to cover the missing role.
6	**Data Concurrency and Race Conditions -** As the backend logic is developed to handle multiple virtual sensors sending data simultaneously, thread-safety issues or database deadlocks may occur in the code, leading to corrupted entries during development testing.
7	**Flawed State Machine Implementation** - Coding logic errors in the shipment lifecycle state machine could allow illegal transitions (e.g., from 'Delivered' back to 'In Transit' status), requiring significant debugging and logic refactoring in the backend.
8	**Incorrect Timezone and Timestamp Handling -** Inconsistent handling of timezones (e.g., mixing local time and UTC) within the backend logic could lead to out-of-order event logging, corrupting the shipment timeline and requiring extensive logic refactoring.
9	**Cross-Platform Mobile Compatibility -** Developing push notifications for supervisors across both iOS and Android platforms may lead to late-discovered incompatibilities, as each operating system handles background processes and notification delivery differently, potentially causing missed or delayed critical alerts.
10	**Fast Changing Regulations -** The system may fail to meet international regulatory and certification standards, blocking real-world deployment.

IMPACT RANK	RISK DESCRIPTION
1	**AI Training Data Scarcity -** Finding an accurate historical dataset of temperature fluctuations to train the AI predictive model might be difficult.
2	**Dependency Conflict & Version Incompatibility** - Conflicts between external libraries (e.g., IoT messaging clients and the core backend framework) during the build process could cause compilation errors, requiring significant time to resolve dependencies before feature development can continue.
3	**Key Personnel Turnover -** A critical team member drops out or becomes unavailable, leaving the rest of the team struggling to cover the missing role.
4	**Inadequate Testing Environment -** It will be difficult to test the system realistically without access to an actual logistics network or constant physical network drops.
5	**Fast Changing Regulations -** The system may fail to meet international regulatory and certification standards, blocking real-world deployment.
6	**Asynchronous Data Handling & Queue Processing Errors -** Developing the backend logic to process high-frequency, continuous data streams asynchronously might lead to out-of-order packet processing, memory leaks, or data loss within the software's message broker/queue architecture.
7	**Data Concurrency and Race Conditions -** As the backend logic is developed to handle multiple virtual sensors sending data simultaneously, thread-safety issues or database deadlocks may occur in the code, leading to corrupted entries during development testing.
8	**Flawed State Machine Implementation** - Coding logic errors in the shipment lifecycle state machine could allow illegal transitions (e.g., from 'Delivered' back to 'In Transit' status), requiring significant debugging and logic refactoring in the backend.
9	**Incorrect Timezone and Timestamp Handling -** Inconsistent handling of timezones (e.g., mixing local time and UTC) within the backend logic could lead to out-of-order event logging, corrupting the shipment timeline and requiring extensive logic refactoring.
10	**Cross-Platform Mobile Compatibility -** Developing push notifications for supervisors across both iOS and Android platforms may lead to late-discovered incompatibilities, as each operating

	system handles background processes and notification delivery differently, potentially causing missed or delayed critical alerts.

LIKELIHOOD RANK	IMPACT RANK	COMBINED RANK	RISK DESCRIPTION
2	1	3	**AI Training Data Scarcity -** Finding an accurate historical dataset of temperature fluctuations to train the AI predictive model might be difficult.
1	2	3	**Dependency Conflict & Version Incompatibility** - Conflicts between external libraries (e.g., IoT messaging clients and the core backend framework) during the build process could cause compilation errors, requiring significant time to resolve dependencies before feature development can continue.
3	4	7	**Inadequate Testing Environment -** It will be difficult to test the system realistically without access to an actual logistics network or constant physical network drops.
5	3	8	**Key Personnel Turnover -** A critical team member drops out or becomes unavailable, leaving the rest of the team struggling to cover the missing role.
4	6	10	**Asynchronous Data Handling & Queue Processing Errors -** Developing the backend logic to process high-frequency, continuous data streams asynchronously might lead to out-of-order packet processing, memory leaks, or data loss within the software's message broker/queue architecture.

6	7	13	**Data Concurrency and Race Conditions -** As the backend logic is developed to handle multiple virtual sensors sending data simultaneously, thread-safety issues or database deadlocks may occur in the code, leading to corrupted entries during development testing.
10	5	15	**Fast Changing Regulations -** The system may fail to meet international regulatory and certification standards, blocking real-world deployment.
7	8	15	**Flawed State Machine Implementation** - Coding logic errors in the shipment lifecycle state machine could allow illegal transitions (e.g., from 'Delivered' back to 'In Transit' status), requiring significant debugging and logic refactoring in the backend.
8	9	17	**Incorrect Timezone and Timestamp Handling -** Inconsistent handling of timezones (e.g., mixing local time and UTC) within the backend logic could lead to out-of-order event logging, corrupting the shipment timeline and requiring extensive logic refactoring.
9	10	19	**Cross-Platform Mobile Compatibility -** Developing push notifications for supervisors across both iOS and Android platforms may lead to late-discovered incompatibilities, as each operating system handles background processes and notification delivery differently, potentially causing missed or delayed critical alerts.